

CV Lab Exam Cheat Sheets (Labs 1-6)

Oct 6, 2026 · @Yesaulah

Index — find your task fast

Core functions were run on OpenCV 4.13 using synthetic test images. Every sheet starts with a THEORY box, then CODE, then TRAPS. Paste `show()` (in the starter cell below) first in any exam notebook.

What "tested" means here

- **Run as written:** all functions and pipelines in 1A-1E, 2A-2F, 3A-3D, 3F, 4B-4E, 5B-5F, 6A-6D, 6F, the starter cell below, and the core of 6E and 6G.
- **Templates, not run verbatim** (file names or points are placeholders you fill in): CT/MRI fusion (2F), 4-point stitching with `[...]` (3E), the full video loops (2G, 6G — logic tested on single frames), `compareHist(h1, h2)` (4D).
- **Parameters were tuned on synthetic images.** Expect to retune on the real lab images (sudoku, smarties, coins, brain MRI): HSV ranges, `param2`, watershed fraction, Canny and threshold values.

STARTER CELL — paste this first in every exam notebook

Later sheets reuse these helpers. With this cell run, every block in the doc works on its own.

```
import cv2, numpy as np, matplotlib.pyplot as plt, pandas as pd

def show(imgs, titles, cols=3, cmap='gray'):
    rows = (len(imgs) + cols - 1) // cols
    plt.figure(figsize=(5*cols, 4*rows))
    for i, (im, t) in enumerate(zip(imgs, titles)):
        plt.subplot(rows, cols, i+1)
        if im.ndim == 3: plt.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB))
        else:           plt.imshow(im, cmap=cmap)
        plt.title(t); plt.axis('off')
    plt.tight_layout(); plt.show()

def load(path, gray=False):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE if gray else
```

```

cv2.IMREAD_COLOR)
    if img is None: raise FileNotFoundError(path)
    return img

def log_transform(g):
    f = g.astype(np.float64)
    c = 255 / np.log(1 + f.max())
    return np.uint8(np.clip(c * np.log1p(f), 0, 255))

def gamma_transform(g, gamma):
    table = np.array([255 * (i / 255) ** gamma for i in
range(256)]).astype(np.uint8)
    return cv2.LUT(g, table)

def gray_world(bgr):
    f = bgr.astype(np.float64)
    avg = f.reshape(-1, 3).mean(axis=0)
    f *= avg.mean() / avg
    return np.uint8(np.clip(f, 0, 255))

print(cv2.__version__)

```

Before the exam: after printing, write the page number next to each section code in the table below (the doc can't print page numbers).

If the question says...	Go to	Key function
load, display, BGR/RGB, subplot, crop, ROI, resize	1A	<code>imread</code> , <code>show()</code>
draw circles/rectangle, blank canvas, text	1B	<code>np.zeros</code> , <code>circle</code> , <code>putText</code>
blend, overlay, transparent caption	1C	<code>addWeighted</code>
mask, cut out shape, bitwise	1D	<code>bitwise_and/not/or</code>
pixel statistics, Pandas	1E	<code>df.describe()</code>
log transform, dark regions	2B	<code>c · log(1+r)</code>
gamma, power-law, midtones	2C	LUT
piecewise, contrast stretching, slicing	2D	<code>np.interp</code>
histogram, equalization, CLAHE	2E	<code>equalizeHist</code>

If the question says...	Go to	Key function
heatmap, colormap, color balance, fusion	2F	<code>applyColorMap</code>
video, webcam, frame loop	2G	<code>VideoCapture</code>
scale, rotate, shear, translate (manual matrix)	3B	<code>warpAffine</code>
rigid, similarity, compose matrices	3C	@ on 3×3
3 points → affine	3D	<code>getAffineTransform</code>
4 points → perspective, bird's-eye, homography (3D / 4×4 / camera projection → 3F)	3E	<code>getPerspectiveTransform</code>
kernel, convolution, box/emboss/sharpen	4B	<code>filter2D</code>
Sobel, Scharr, gradient magnitude/direction	4C	<code>Sobel</code> , <code>cartToPolar</code>
HOG, LBP, color/edge histogram features	4D	<code>skimage.feature</code>
Canny, LoG, Laplacian	4E / 6A	<code>Canny</code> , <code>Laplacian</code>
global / adaptive / Otsu threshold	5B	<code>threshold</code> , <code>adaptiveThreshold</code>
color segmentation, HSV range	5C	<code>inRange</code>
region growing, seed point	5D	<code>region_growing()</code>
touching coins, watershed, distance transform	5E	<code>watershed</code>
K-means, clustering, color quantization	5F	<code>kmeans</code>
lines, lanes, screen edges	6B	<code>HoughLinesP</code>
circles, coin counting	6C	<code>HoughCircles</code>
SIFT keypoints, matching, object recognition	6D	<code>SIFT_create</code> , <code>BFMatcher</code>
panorama, stitching	6E	<code>findHomography</code>
wavelet, denoise, sensor anomaly	6F	<code>pywt.wavedec</code>
contours, boundary, security zone	6G	<code>findContours</code>

Sheet 1 — Basics & I/O (Lab 1)

Needs: nothing — 1A defines the imports and `show()` (also in the starter cell).

1-THEORY

- **Digital image:** 2-D function $f(x, y)$; x = row ($0 \dots M-1$), y = column ($0 \dots N-1$); value = intensity/gray level. All of x, y, f finite and discrete $\rightarrow$ digital image.
- **Grayscale:** 1 channel, 0 = black, 255 = white (uint8). **Color:** 3 channels. **OpenCV stores BGR**, Matplotlib expects RGB.
- **Resolution:** width $\times$ height in pixels. `img.shape` = (height, width, channels).
- **11 fundamental DIP steps (in order):** 1 Acquisition (first, includes scaling) $\rightarrow$ 2 Enhancement (simplest, highlights features) $\rightarrow$ 3 Restoration (improves appearance, model-based) $\rightarrow$ 4 Color processing $\rightarrow$ 5 Wavelets & multi-resolution $\rightarrow$ 6 Compression (reduce storage) $\rightarrow$ 7 Morphological processing (extract shape components) $\rightarrow$ 8 Segmentation (partition into objects; hardest step) $\rightarrow$ 9 Representation & description $\rightarrow$ 10 Object recognition (assign label from descriptors) $\rightarrow$ 11 Knowledge base (last).
- **5 purposes of DIP:** visualization, sharpening & restoration, retrieval, measurement of pattern, recognition.
- **Terms:** pixel = smallest unit; histogram = distribution of intensities; homography = maps two images of the same plane; optical flow = motion between frames; feature descriptor = compact vector used for matching; SIFT = scale-invariant features; SURF = faster SIFT-like; HOG = gradient-orientation descriptor for detection.

Library	Focus	Language	Abstraction	Note
OpenCV	General CV	C++, Python	High	Strong community, DL bindings
scikit-image	Image processing	Python	Medium	Integrates with NumPy/Matplotlib
Pillow	Basic image I/O	Python	Low	PIL fork
TorchVision	CV for PyTorch	Python	High	Native PyTorch
TensorFlow / Keras	ML incl. CV	Python	High	Keras = high-level API in TF
OpenVINO	Inference optimization	C++, Python	High	Intel

Library	Focus	Language	Abstraction	Note
Caffe	DL framework	C++	Medium	CNN focus
Detectron2	Object detection	Python	High	Built on PyTorch
Hugging Face	NLP + CV models	Python	High	Pretrained models

1A — Load, check, display (paste this first)

```
import cv2, numpy as np, matplotlib.pyplot as plt, pandas as pd

def show(imgs, titles, cols=3, cmap='gray'):
    rows = (len(imgs) + cols - 1) // cols
    plt.figure(figsize=(5*cols, 4*rows))
    for i, (im, t) in enumerate(zip(imgs, titles)):
        plt.subplot(rows, cols, i+1)
        if im.ndim == 3: plt.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB))
        else:            plt.imshow(im, cmap=cmap)
        plt.title(t); plt.axis('off')
    plt.tight_layout(); plt.show()

img = cv2.imread('image.jpg') # BGR
if img is None: raise FileNotFoundError('Image not found')
gray = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE) # or cvtColor(img,
cv2.COLOR_BGR2GRAY)
h, w = img.shape[:2]

small = cv2.resize(img, None, fx=0.5, fy=0.5) # by factor
fixed = cv2.resize(img, (500, 500)) # (WIDTH, HEIGHT)
blur = cv2.GaussianBlur(img, (25, 25), 0) # odd kernel; sigma 0 =
auto
roi = img[h//2-150:h//2+150, w//2-150:w//2+150] # [rows, cols] = [y, x];
300x300 center
left = img[:, :w//2] # left half

show([img, blur, roi], ['Original', 'Blur', 'ROI'])
# Formatted title: plt.title('King', fontsize=16, color='darkred', pad=15)
```

1B — Blank canvas, drawing, text

```

canvas = np.zeros((800, 800, 3), np.uint8)           # black 800x800
c = (400, 400)                                       # (x, y)
for i, r in enumerate([250, 200, 150, 100, 50]):    # big to small, filled
    color = (0,0,255) if i % 2 == 0 else (255,255,255)
    cv2.circle(canvas, c, r, color, -1)             # -1 = filled, else
thickness
cv2.rectangle(canvas, (150,150), (650,650), (0,255,0), 3) # box around r=250
cv2.line(canvas, (0,0), (799,799), (255,0,0), 2)
cv2.fillPoly(canvas, [np.array([[400,80],[600,500],[200,500]]], np.int32)],
(0,255,255))
cv2.putText(canvas, 'Center', (330,790), cv2.FONT_HERSHEY_SIMPLEX, 1.5,
(255,255,0), 2)
#          img,    text,    bottom-left(x,y), font,          scale, color(BGR),
thickness
print('Center:', (canvas.shape[1]//2, canvas.shape[0]//2))

```

1C — Blending, semi-transparent caption

```

overlay = img.copy()
cv2.rectangle(overlay, (0, int(h*0.8)), (w, h), (255, 0, 0), -1) # bottom
20% blue
caption = cv2.addWeighted(overlay, 0.4, img, 0.6, 0)                # a*img1
+ b*img2 + gamma
cv2.putText(caption, 'Title', (20, h-20), cv2.FONT_HERSHEY_SIMPLEX, 1,
(255,255,255), 2)

img2 = cv2.resize(cv2.imread('image2.jpg'), (w, h)) # sizes MUST match
added = cv2.add(img, img2)                                # saturating sum
(brighter)

```

1D — Masks and bitwise

```
a = cv2.resize(cv2.imread('image.jpg'), (500, 500))
b = cv2.resize(cv2.imread('image2.jpg'), (500, 500))
mask = np.zeros((500, 500), np.uint8) # mask = 1 channel uint8
cv2.circle(mask, (250, 250), 150, 255, -1) # white shape
fg = cv2.bitwise_and(a, a, mask=mask) # keep a inside shape
bg = cv2.bitwise_and(b, b, mask=cv2.bitwise_not(mask)) # keep b outside
combo = cv2.bitwise_or(fg, bg)
show([mask, fg, bg, combo], ['Mask', 'FG', 'BG', 'Combined'], cols=4)
```

1E — Pixel statistics with Pandas, thresholds, rotation

```
B, G, R = cv2.split(img)
df = pd.DataFrame({'Red': R.flatten(), 'Green': G.flatten(), 'Blue':
B.flatten()})
print(df.describe()) # count, mean, std, min,
25/50/75%, max

_, th = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY) # (src,
thresh, maxval, type)
ad = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY, 11, 2)
M = cv2.getRotationMatrix2D((w/2, h/2), 45, 0.8) # center, angle (+ = CCW),
scale
rot = cv2.warpAffine(ad, M, (w, h)) # dsize = (WIDTH, HEIGHT)
eq = cv2.equalizeHist(gray) # gray only
```

1-TRAPS

- `img.shape` → (h, w). But `resize`, `warpAffine`, `warpPerspective` take (w, h). Points are (x, y); slicing is [y, x].
- `cv2.add(a, b)`: $250 + 10 = 255$ (saturates). NumPy `a + b` on uint8: $250 + 10 = 4$ (wraps). `cv2.subtract(10, 250) = 0`, NumPy gives 16. Tested.
- Matplotlib on a BGR image shows red/blue swapped. Grayscale needs `cmap='gray'`.
- `cv2.imshow` does not render in Jupyter; use `show()`. In a .py script: `cv2.imshow(...); cv2.waitKey(0); cv2.destroyAllWindows()`.
- `cv2.threshold` returns **two** values: `ret, out`.
- Gaussian kernel sizes must be **odd**. Thickness `-1` = filled.

- Lab 1 manual's threshold explanation says 100, code says 190: in `threshold(src, T, maxval, type)` pixels $> T$ become maxval.

Sheet 2 — Point / Photometric transforms (Lab 2)

Needs: starter cell. 2G (video) uses `log_transform`, `gamma_transform`, `gray_world` — all in the starter cell.

2-THEORY

- **Point (photometric) transform:** output pixel depends only on the SAME input pixel: $s = T(r)$. Neighbours don't matter. Contrast: spatial filters (Sheet 4) use a neighbourhood. $L = 256$ levels, r and s in $[0, L-1]$.
- Implemented fastest as a **lookup table (LUT)** of 256 entries.

Transform	Formula	Effect	Use when
Negative	$s = L - 1 - r$	Inverts	White/gray detail in dark regions (mammograms)
Log	$s = c \cdot \log(1 + r)$, $c = 255 / \log(1 + r_{\max})$	Expands DARK values, compresses bright	Underexposed image, huge dynamic range (Fourier spectrum)
Inverse log	$s = \exp(r/c) - 1$	Expands bright	Opposite of log
Gamma (power-law)	$s = c \cdot r^\gamma$ (r normalised to 0-1, then $\times 255$)	$\gamma < 1$ BRIGHTENS (like log); $\gamma > 1$ DARKENS; $\gamma = 1$ identity	Monitor gamma correction, fine-tune midtones, suppress glare ($\gamma > 1$)
Contrast stretching (piecewise)	Line segments through $(0,0)$, (r_1,s_1) , (r_2,s_2) , $(255,255)$	Expands range $r_1 \dots r_2$	Low-contrast image
Thresholding (piecewise limit)	$r_1 = r_2$, $s_1 = 0$, $s_2 = 255$	Binary	Special case of stretching
Min-max stretch	$s = (r - r_{\min}) \cdot 255 / (r_{\max} - r_{\min})$	Full 0-255 range	Uses image's own min/max

Transform	Formula	Effect	Use when
Intensity-level slicing	Range $[A,B] \rightarrow 255$; rest $\rightarrow 0$ (or unchanged)	Highlights a band	Masses of water, flaws
Bit-plane slicing	Plane $k = (r \gg k) \& 1$	Bit 7 = most significant	Compression analysis

- **Piecewise rules:** $r_1 = s_1$ and $r_2 = s_2 \rightarrow$ identity (no change). $r_1 = r_2$, $s_1 = 0$, $s_2 = L-1 \rightarrow$ thresholding. Need $r_1 \leq r_2$ and $s_1 \leq s_2$ so the function stays monotonic.
- **Histogram:** $h(r_k) = n_k$ (count of pixels at level k). Normalised $p(r_k) = n_k / MN$ (a probability). Dark image $\rightarrow$ bins on the left; bright $\rightarrow$ right; low contrast $\rightarrow$ narrow; high contrast $\rightarrow$ spread across.
- **Histogram equalization:** $s_k = (L-1) \cdot \sum_{j=0..k} p(r_j) = (L-1) \cdot \text{CDF}(r_k)$. Spreads intensities toward a flat (uniform) histogram, **automatically**, no parameters. Global. Drawbacks: amplifies noise; can over-brighten; result is only approximately flat (discrete levels).
- **CLAHE** (Contrast Limited Adaptive HE): equalizes per tile (e.g., 8×8) and clips each histogram at `clipLimit` before computing the CDF, so noise isn't amplified. Tiles joined with bilinear interpolation.
- **Equalizing colour images:** never equalize B, G, R separately (changes colours). Convert to YCrCb/HSV/LAB, equalize only the brightness channel (Y / V / L).
- **Colormap (false colour/pseudocolour):** maps gray intensities to colours (JET: blue = low, red = high). The eye separates colours better than ~ 30 gray shades, so subtle boundaries become visible. Adds no new information.
- **Color balance (gray-world):** assumes the average scene colour is gray; scale each channel so its mean equals the overall mean $\rightarrow$ removes colour cast.
- **Image fusion:** weighted sum $\alpha \cdot A + \beta \cdot B + \gamma$ (`addWeighted`). Higher weight on the modality whose information you want to dominate (CT = edges/bone, MRI = soft tissue).

2A — Plain transforms (all return uint8)

```
import cv2, numpy as np, matplotlib.pyplot as plt
gray = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

def negative(g):
    return 255 - g

def bit_plane(g, k):
    # k = 0 (LSB) ... 7 (MSB)
    return ((g >> k) & 1) * 255
```

2B — Log transform

```
def log_transform(g):
    f = g.astype(np.float64)          # float first, else overflow
    c = 255 / np.log(1 + f.max())
    return np.uint8(np.clip(c * np.log1p(f), 0, 255))  # log1p(x) = log(1+x)

log_img = log_transform(gray)
```

2C — Gamma/power-law

```
def gamma_transform(g, gamma):
    # works on gray AND colour
    table = np.array([255 * (i / 255) ** gamma for i in
range(256)]).astype(np.uint8)
    return cv2.LUT(g, table)

bright = gamma_transform(gray, 0.5)    #  $\gamma < 1$  brighter (tested: mean 74 → 133)
dark   = gamma_transform(gray, 2.0)    #  $\gamma > 1$  darker   (tested: mean 74 → 30)
show([gray, log_img, bright, dark], ['Original', 'Log', ' $\gamma=0.5$ ', ' $\gamma=2.0$ '],
cols=4)
# Compare several: for gm in [0.3, 0.5, 1, 1.5, 2.5]: ...
```

2D — Piecewise/contrast stretching/slicing

```
def contrast_stretch(g, r1, s1, r2, s2):
    table = np.interp(np.arange(256), [0, r1, r2, 255], [0, s1, s2,
255]).astype(np.uint8)
    return cv2.LUT(g, table)
```

```

def minmax_stretch(g):
    return cv2.normalize(g, None, 0, 255, cv2.NORM_MINMAX)

def slice_levels(g, A, B, keep_background=True):
    out = g.copy() if keep_background else np.zeros_like(g)
    out[(g >= A) & (g <= B)] = 255
    return out

cs = contrast_stretch(gray, 70, 20, 180, 235)

# Plot the transfer curve (marks for 'show the function')
x = np.arange(256)
plt.plot(x, np.interp(x, [0,70,180,255], [0,20,235,255]), label='stretch')
plt.plot(x, 255*(x/255)**0.5, label='γ=0.5')
plt.plot(x, (255/np.log(256))*np.log1p(x), label='log')
plt.xlabel('input r'); plt.ylabel('output s'); plt.legend(); plt.show()

```

2E — Histograms, equalization, CLAHE

```

hist = cv2.calcHist([gray], [0], None, [256], [0, 256]) # [img], [channel],
mask, [bins], [range]
plt.plot(hist); plt.xlim([0, 256]); plt.show()
# or: plt.hist(gray.ravel(), bins=256, range=(0, 256))

img = cv2.imread('image.jpg')
for i, col in enumerate(['b', 'g', 'r']): # colour histogram
    plt.plot(cv2.calcHist([img], [i], None, [256], [0, 256]), color=col)
plt.show()

eq = cv2.equalizeHist(gray) # gray uint8 only
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
cl = clahe.apply(gray)

def equalize_manual(g): # if asked to implement
    hist, _ = np.histogram(g.flatten(), 256, [0, 256])
    cdf = hist.cumsum()
    cdf_m = np.ma.masked_equal(cdf, 0)
    cdf_m = (cdf_m - cdf_m.min()) * 255 / (cdf_m.max() - cdf_m.min())
    table = np.ma.filled(cdf_m, 0).astype(np.uint8)
    return table[g] # matches equalizeHist
within 1 level

```

```

ycrcb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)      # colour equalization
ycrcb[:, :, 0] = cv2.equalizeHist(ycrcb[:, :, 0])
eq_color = cv2.cvtColor(ycrcb, cv2.COLOR_YCrCb2BGR)

# Before/after with histograms
plt.figure(figsize=(12, 6))
for i, (im, t) in enumerate([(gray, 'Original'), (eq, 'Equalized'), (c1,
'CLAHE')]):
    plt.subplot(2, 3, i+1); plt.imshow(im, cmap='gray'); plt.title(t);
plt.axis('off')
    plt.subplot(2, 3, i+4); plt.hist(im.ravel(), bins=256, range=(0, 256))
plt.tight_layout(); plt.show()

```

2F — Colormap, color balance, threshold, fusion

```

heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET)      # input: 1-channel uint8
→ BGR output
# others: COLORMAP_HOT, COLORMAP_BONE, COLORMAP_INFERNO

def gray_world(bgr):                                # colour balance
    f = bgr.astype(np.float64)
    avg = f.reshape(-1, 3).mean(axis=0)              # mean B, G, R
    f *= avg.mean() / avg
    return np.uint8(np.clip(f, 0, 255))
balanced = gray_world(heat)
bright_contrast = cv2.convertScaleAbs(img, alpha=1.2, beta=20) #  $s = \alpha \cdot r + \beta$ 

_, dense = cv2.threshold(eq, 200, 255, cv2.THRESH_BINARY) # keep only
densest tissue
dense_only = cv2.bitwise_and(eq, eq, mask=dense)

# Multi-modal fusion (CT + MRI, same size)
ct = cv2.imread('ct.png', 0); mri = cv2.imread('mri.png', 0)
mri = cv2.resize(mri, (ct.shape[1], ct.shape[0]))
ct_e, mri_e = cv2.equalizeHist(ct), cv2.equalizeHist(mri)
ct_c = cv2.cvtColor(ct_e, cv2.COLOR_GRAY2BGR)
mri_c = cv2.applyColorMap(mri_e, cv2.COLORMAP_JET)
fused = cv2.addWeighted(ct_c, 0.7, mri_c, 0.3, 0)    # heavier CT weight
final = gamma_transform(log_transform(cv2.cvtColor(fused,
cv2.COLOR_BGR2GRAY)), 1.5)
show([ct, mri, fused, final], ['CT', 'MRI', 'Fused', 'Log+Gamma'], cols=4)

```

2G — Video processing loop

```
cap = cv2.VideoCapture('echo.mp4')                # 0 = webcam
while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break                               # end of video
    g = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    g = cv2.equalizeHist(g)
    g = log_transform(g)
    g = gamma_transform(g, 1.5)                     #  $\gamma > 1$  suppresses bright
noise                                                noise
    enh = gray_world(cv2.applyColorMap(g, cv2.COLORMAP_JET))
    combined = np.hstack([frame, enh])              # same height &
channels!                                           channels!
    cv2.imshow('Raw | Enhanced', combined)
    if cv2.waitKey(25) & 0xFF == ord('q'): break
cap.release(); cv2.destroyAllWindows()
# Jupyter: replace imshow with saving a few frames, then show([...])
# Save video: out = cv2.VideoWriter('out.mp4',
cv2.VideoWriter_fourcc(*'mp4v'), 25, (W, H)); out.write(f); out.release()
```

2-TRAPS

- Gamma: $\gamma < 1$ **brightens**, $\gamma > 1$ darkens (with r normalised to 0-1). Log behaves like $\gamma < 1$.
- Always compute $\log/\gamma$ in **float**, then clip and cast to uint8. `np.log(0) = -inf`, hence $\log(1+r)$.
- `equalizeHist` accepts only 1-channel uint8. `applyColorMap` needs uint8.
- `np.hstack` needs equal heights and equal channel counts: convert gray to BGR first with `cv2.cvtColor(g, cv2.COLOR_GRAY2BGR)`.
- `calcHist` arguments are lists: `[img], [0], [256], [0, 256]`.
- `plt.hist` range must be a keyword: `range=(0, 256)`.

Sheet 3 — Geometric transforms (Lab 3)

Needs: starter cell + 3A (matrix builders `T`, `S`, `Sh`, `R`, `apply`, `about_center`) before 3B-3E. 3F is self-contained.

3-THEORY

- **Geometric transform:** moves pixel POSITIONS $(x, y) \rightarrow (x', y')$; intensities unchanged (compare Sheet 2, which changes values). Output pixels are filled by interpolation (nearest, bilinear `INTER_LINEAR` default, bicubic).
- **2×2 matrix $[[a, b], [c, d]]$:** column 1 = where the x-axis unit vector lands, column 2 = where the y-axis unit vector lands. a, d = scale; b, c = shear/rotation.
- **Linear transforms (2×2)** — scale, rotate, shear, reflect. Origin always maps to origin (**anchored origin**), so they CANNOT translate.
- **Homogeneous coordinates:** write (x, y) as $(x, y, 1)$. A 3×3 matrix can now hold translation in its 3rd column. Transforms compose by matrix multiplication; the **rightmost matrix is applied first**.
- **Projective (homography):** bottom row $[g, h, 1]$ is not $[0, 0, 1]$. Then $x' = (ax + by + c)/(gx + hy + 1)$, $y' = (dx + ey + f)/(gx + hy + 1)$. This **denominator** makes far points shrink $\rightarrow$ perspective. Parallel lines may converge.
- **Determinant:** $\det(2 \times 2 \text{ part})$ = area scale factor; $\det < 0$ = reflection; rotation $\det = 1$.

Transform	Matrix (3×3)	DOF	Preserves	Points needed
Translation	$[[1, 0, t_x], [0, 1, t_y], [0, 0, 1]]$	2	Everything except position	1
Rigid / Euclidean	$[[\cos, -\sin, t_x], [\sin, \cos, t_y], [0, 0, 1]]$	3	Lengths, angles, area	2
Similarity	$[[s \cdot \cos, -s \cdot \sin, t_x], [s \cdot \sin, s \cdot \cos, t_y], [0, 0, 1]]$	4	Angles, length RATIOS, shape	2
Affine	$[[a, b, c], [d, e, f], [0, 0, 1]]$	6	Parallel lines, straight lines, ratio along a line	3
Projective / Homography	$[[a, b, c], [d, e, f], [g, h, 1]]$	8	Straight lines only (collinearity)	4

Linear-only matrices (2×2, origin fixed): Scale $[[s_x, 0], [0, s_y]]$ · Rotation $[[\cos\theta, -\sin\theta], [\sin\theta, \cos\theta]]$ · Shear-x $[[1, sh_x], [0, 1]]$ · Shear-y $[[1, 0], [sh_y, 1]]$ · Reflect-x $[[1, 0], [0, -1]]$.

Hierarchy (each contains the ones above): Translation $\subset$ Rigid $\subset$ Similarity $\subset$ Affine $\subset$ Projective.

3A — Matrix builders (paste once)

```
import cv2, numpy as np
img = cv2.imread('image.jpg'); h, w = img.shape[:2]; cx, cy = w/2, h/2

def T(tx, ty):          return np.array([[1,0,tx],[0,1,ty],[0,0,1]],
np.float64)
def S(sx, sy):          return np.array([[sx,0,0],[0,sy,0],[0,0,1]],
np.float64)
def Sh(shx=0, shy=0):  return np.array([[1,shx,0],[shy,1,0],[0,0,1]],
np.float64)
def R(deg):             # textbook matrix: CLOCKWISE on screen (y points down)
    t = np.deg2rad(deg); c, s = np.cos(t), np.sin(t)
    return np.array([[c,-s,0],[s,c,0],[0,0,1]], np.float64)

def apply(img, M3, size=None):          # size = (WIDTH, HEIGHT)
    h, w = img.shape[:2]
    return cv2.warpAffine(img, M3[:2].astype(np.float32), size or (w, h))

def about_center(M, cx, cy):            # do M around the image centre
    return T(cx, cy) @ M @ T(-cx, -cy)
```

3B — Scale, rotate, shear, translate

```
# SCALE 300% and keep centred on a 3x bigger canvas
s = 3; W, H = int(w*s), int(h*s)
big = apply(img, T(W/2, H/2) @ S(s, s) @ T(-cx, -cy), (W, H))
zoom = apply(img, about_center(S(3, 3), cx, cy))          # same canvas =
zoom into centre
# built-in: cv2.resize(img, None, fx=3, fy=3, interpolation=cv2.INTER_LINEAR)

# ROTATE without cutting corners: new canvas size
ang = 45                                                    # + = counter-
clockwise below
t = np.deg2rad(ang); c, s_ = abs(np.cos(t)), abs(np.sin(t))
nw, nh = int(w*c + h*s_), int(w*s_ + h*c)                  # new_w =
w|cos|+h|sin|
rot = apply(img, T(nw/2, nh/2) @ R(-ang) @ T(-cx, -cy), (nw, nh))
# same with OpenCV (tested identical):
M = cv2.getRotationMatrix2D((cx, cy), ang, 1.0)            # + = CCW
M[0, 2] += nw/2 - cx; M[1, 2] += nh/2 - cy
rot2 = cv2.warpAffine(img, M, (nw, nh))
```

```
# 'Spin back' an image rotated by +45: use -45 (or the other sign); check
visually.

# SHEAR in x; widen canvas by shx*h
shx = 0.3
sheared = apply(img, Sh(shx, 0), (int(w + shx*h), h))
unsheared = apply(img, Sh(-shx, 0)) # undo slant:
negative shear

# TRANSLATE: content is 150 left & 80 up → move +150 right, +80 down
moved = apply(img, T(150, 80))
```

3C — Rigid & similarity (one combined 3×3)

```
rigid = T(100, 50) @ about_center(R(-30), cx, cy) # rotate
then shift, scale = 1
sim = T(100, 50) @ about_center(R(-30) @ S(1.5, 1.5), cx, cy) # + uniform
scale
print(np.round(sim, 3)) # print
the single matrix
out_r, out_s = apply(img, rigid), apply(img, sim)
# check rigid: det(rigid[:2,:2]) == 1 ; similarity: det == s**2
```

3D — General affine from 3 point pairs

```
src = np.float32([[50,50], [200,50], [50,200]]) # (x, y) before
dst = np.float32([[10,100], [200,50], [100,250]]) # (x, y) after
A = cv2.getAffineTransform(src, dst) # 2x3

# Manual: solve 6 unknowns from 6 equations (tested equal)
rows, b = [], []
for (x, y), (u, v) in zip(src, dst):
    rows += [[x, y, 1, 0, 0, 0], [0, 0, 0, x, y, 1]]; b += [u, v]
A_manual = np.linalg.solve(np.array(rows, float), np.array(b,
float)).reshape(2, 3)

warped = cv2.warpAffine(img, A, (w, h))
repaired = cv2.warpAffine(img, cv2.invertAffineTransform(A), (w, h)) # undo
a known distortion
# or map the corrupted -> correct points directly:
getAffineTransform(corrupt_pts, correct_pts)
```


3E — Perspective / homography (4 points)

```
# Bird's-eye view: 4 corners in TL, TR, BR, BL order
src = np.float32([[100,80], [500,60], [560,380], [40,360]])
W, H = 400, 400
dst = np.float32([[0,0], [W-1,0], [W-1,H-1], [0,H-1]])
P = cv2.getPerspectiveTransform(src, dst) # 3x3
bird = cv2.warpPerspective(img, P, (W, H))

# Insert a flat painting into an angled frame
painting = cv2.resize(cv2.imread('painting.jpg'), (300, 200))
ph, pw = painting.shape[:2]
corners = np.float32([[0,0], [pw,0], [pw,ph], [0,ph]])
frame_pts = np.float32([[120,60], [420,100], [410,330], [130,360]])
Hm = cv2.getPerspectiveTransform(corners, frame_pts)
warped = cv2.warpPerspective(painting, Hm, (w, h))
mask = np.zeros((h, w), np.uint8)
cv2.fillConvexPoly(mask, frame_pts.astype(np.int32), 255)
result = img.copy(); result[mask == 255] = warped[mask == 255]

# Stitch 2 photos from 4 matching points (img2 → img1 space)
pts2 = np.float32([[x1,y1], [x2,y2], [x3,y3], [x4,y4]]) # in img2
pts1 = np.float32([[...], [...], [...], [...]]) # same points in
img1
Hs, _ = cv2.findHomography(pts2, pts1) # 4 pts exact; more
pts → add cv2.RANSAC
pano = cv2.warpPerspective(img2, Hs, (img1.shape[1] + img2.shape[1],
img1.shape[0]))
pano[:img1.shape[0], :img1.shape[1]] = img1

# Map points through a homography
new_pts = cv2.perspectiveTransform(pts2.reshape(-1, 1, 2), Hs)
# Find point coordinates: plt.imshow(img) shows (x, y) on hover; or
plt.ginput(4) in a GUI backend
```

3F — 3D transformations (4×4) and projection to 2D

- **2D vs 3D:** 2D works on $(x, y) \rightarrow$ image plane, photo, document, planar object; matrices 2×3 (`warpAffine`) or 3×3 homogeneous (`warpPerspective`). **3D** works on $(x, y, z) \rightarrow$ robotics, 3D graphics; homogeneous point $(x, y, z, 1)$ and a 4×4 matrix $[[R, t], [0, 0, 0, 1]]$ ($R = 3 \times 3$ rotation, $t = 3 \times 1$ translation).

- **Image coordinate system:** origin (0, 0) at **top-left**, $x \rightarrow$ right, $y \rightarrow$ **down**. Last pixel of a 640×480 image = (639, 479). Points are written (x, y) but arrays are indexed `image[y, x]`.
- **Geometric vs photometric:** geometric changes WHERE a pixel is (rotation, scaling, shear, translation, perspective); photometric changes WHAT it looks like (brightness, contrast, gamma, colour conversion).
- **3D world $\rightarrow$ 2D image (pinhole camera):** $u = f \cdot X/Z + cx$, $v = f \cdot Y/Z + cy$. Dividing by depth Z makes far objects smaller (same idea as the homography denominator). Full camera projection/calibration is a separate topic.
- **3D DOF:** translation 3, rotation 3 (about x, y, z), rigid 6.

```
def T3(tx, ty, tz): M = np.eye(4); M[:3, 3] = [tx, ty, tz]; return M
def S3(sx, sy, sz): return np.diag([sx, sy, sz, 1.0])
def Rx(d):
    t = np.deg2rad(d); c, s = np.cos(t), np.sin(t)
    return np.array([[1,0,0,0], [0,c,-s,0], [0,s,c,0], [0,0,0,1]])
def Ry(d):
    t = np.deg2rad(d); c, s = np.cos(t), np.sin(t)
    return np.array([[c,0,s,0], [0,1,0,0], [-s,0,c,0], [0,0,0,1]])
def Rz(d):
    t = np.deg2rad(d); c, s = np.cos(t), np.sin(t)
    return np.array([[c,-s,0,0], [s,c,0,0], [0,0,1,0], [0,0,0,1]])

P = np.array([1, 0, 0, 1.0])          # homogeneous 3D point
print(Rz(90) @ P)                    #  $\rightarrow$  (0, 1, 0, 1)
print(T3(1, 2, 3) @ S3(2, 2, 2) @ P) # scale then translate  $\rightarrow$  (3, 2, 3, 1)

# Project a unit cube to the image
cube = np.array([[x, y, z, 1] for x in (0, 1) for y in (0, 1) for z in (0, 1)], float).T # 4x8
moved = T3(0, 0, 5) @ Ry(30) @ cube      # rotate, then push 5 units in
front of camera
f, cx, cy = 500, 320, 240
u = f * moved[0] / moved[2] + cx
v = f * moved[1] / moved[2] + cy
# OpenCV equivalent (tested identical):
K = np.array([[f, 0, cx], [0, f, cy], [0, 0, 1.0]])
pts2d, _ = cv2.projectPoints(cube[:3].T, np.deg2rad([0, 30, 0]), np.array([0, 0, 5.0]), K, None)
```

3-TRAPS

- Composition order: `T2 @ R @ T1` applies **T1 first**. To act around the centre: translate centre → origin, transform, translate back.
- Textbook $R(+\theta)$ turns **clockwise** on screen; `getRotationMatrix2D(+\theta)` turns **counter-clockwise** (it equals $R(-\theta)$). Tested.
- `warpAffine` needs 2×3 float32 → pass `M3[:2]`. `warpPerspective` takes the full 3×3 .
- Point arrays for `getAffineTransform` / `getPerspectiveTransform` must be **float32**, exactly 3 / 4 points, same order in src and dst.
- Corners cut off after scale/rotate/shear = output canvas too small; enlarge `dsize` and add translation.
- Black regions after warping are normal (no source pixel). Use `borderMode=cv2.BORDER_REPLICATE` to fill.

Sheet 4 — Filtering, features, edges (Lab 4)

Needs: starter cell + 4A (skimage imports). 4D's HED/HIG use `mag`, `ang` from 4C — run 4C first.

4-THEORY

- **Feature:** a measurable, distinctive property (edge, corner, blob, texture, colour). Types: **global** (whole image, e.g., colour histogram) vs **local** (around keypoints, e.g., SIFT); **low-level** (edges, corners) vs **high-level** (shapes, objects). Good features: repeatable, distinctive, invariant (scale/rotation/illumination), efficient.
- **Feature extraction techniques:** edge-based, corner (Harris), blob, texture, histogram-based, learned (CNN).

Descriptor	Captures	How it works
HOG	Shape via gradient orientation	Gradients → cells (8×8) → orientation histogram (9 bins, $0-180^\circ$) per cell, weighted by magnitude → normalise over blocks (2×2 cells) → concatenate. Classic for pedestrian detection.

Descriptor	Captures	How it works
LBP	Texture, illumination-robust	Compare each of P neighbours (radius R) with centre: $\geq$ centre $\rightarrow$ 1 else 0 $\rightarrow$ P-bit binary number $\rightarrow$ histogram of codes. "Uniform" = $\leq$ 2 bit transitions $\rightarrow$ P+2 bins.
Colour histogram	Colour distribution, rotation-invariant, no spatial info	Count pixels per colour bin (often HSV H×S), normalise.
HED (edge directions)	Dominant edge orientations	Canny edge pixels $\rightarrow$ gradient angle $\rightarrow$ histogram of angles at edges only.
HIG (intensity gradients)	Overall gradient orientation	Angle histogram over ALL pixels, weighted by magnitude.
Texture energy / contrast	Local roughness	Energy = local mean of I^2 ; contrast = local std dev, over $n \times n$ windows.

- **Filtering vs convolution:** correlation slides the kernel as-is; true **convolution flips the kernel 180°** first. `cv2.filter2D` does CORRELATION. Identical for symmetric kernels.
- **Output size:** $O = \lfloor (N - K + 2P) / S \rfloor + 1$ (N input, K kernel, P padding, S stride).
 - **Valid** (P = 0): shrinks by K-1. **Same** (P = (K-1)/2): same size. **Strided** (S = 2): roughly halves.
- **Smoothing:** box/mean (equal weights, sum = 1), Gaussian (weights by distance, best general), median (non-linear, best for **salt-and-pepper**), bilateral (preserves edges).
- **Sharpening** = original + high-pass. Kernel weights sum to 1. **Edge** kernels sum to 0.
- **Gradient:** G_x, G_y from first derivative. Magnitude = $\sqrt{G_x^2 + G_y^2}$ (approx $|G_x| + |G_y|$); direction $\theta = \text{atan2}(G_y, G_x)$. Gradient direction is **perpendicular** to the edge.
- **Edge detectors:** Roberts (2×2, diagonal, noise-sensitive), Prewitt (3×3, equal weights), **Sobel** (3×3, centre weight 2 $\rightarrow$ some smoothing), **Scharr** (more accurate 3×3 rotation symmetry: 3, 10, 3), Laplacian (2nd derivative, isotropic, very noise-sensitive, zero-crossings), Canny (best/optimal).

Kernel	Matrix
Sobel x	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

Kernel	Matrix
Sobel y	$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$
Prewitt x	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$
Scharr x	$\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$
Roberts	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ and $\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$
Laplacian	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ (8-neighbour: centre -8)
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$
Emboss	$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$
Box 3×3	$\text{ones}(3,3) / 9$

- **Canny — 4 steps in order:** 1) **Gaussian smoothing** (reduce noise) → 2) **Gradient magnitude & direction** (Sobel) → 3) **Non-maximum suppression** (keep only local maxima along gradient direction → thin 1-pixel edges) → 4) **Hysteresis** with two thresholds: > high = strong edge (kept); < low = discarded; between = weak, kept **only if connected** to a strong edge.
 - Raise **high** → fewer strong seeds → missing edges. Lower **low** → more weak edges connect → noisier. Typical ratio high:low = 2:1 or 3:1. Parameters: low, high, aperture (Sobel size), L2gradient.

4A — Setup

```
import cv2, numpy as np, matplotlib.pyplot as plt
from skimage.feature import hog, local_binary_pattern
from skimage import exposure
img = cv2.imread('image.jpg'); gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

4B — Filtering & convolution

```
box    = cv2.filter2D(img, -1, np.ones((3, 3), np.float32) / 9)    # -1 = same
depth  as input
box2   = cv2.blur(img, (3, 3))                                     #
identical (tested)
g1d    = cv2.getGaussianKernel(5, 1.0)                             # 5x1
column!
gauss  = cv2.filter2D(img, -1, g1d @ g1d.T)                         # make it
```

```

5x5
gauss2 = cv2.GaussianBlur(img, (5, 5), 1.0)
median = cv2.medianBlur(img, 5) # salt &
pepper
bilat = cv2.bilateralFilter(img, 9, 75, 75) # edge-
preserving
sharp = cv2.filter2D(img, -1, np.array([[0,-1,0],[-1,5,-1],[0,-1,0]],
np.float32))
emboss = cv2.filter2D(gray, -1, np.array([[-2,-1,0],[-1,1,1],[0,1,2]],
np.float32))

# Convolution types (CV_32F output keeps negative values)
k = np.array([[1,0,-1],[1,0,-1],[1,0,-1]], np.float32)
same = cv2.filter2D(gray, cv2.CV_32F, k, borderType=cv2.BORDER_CONSTANT) #
zero padding, same size
valid = same[1:-1, 1:-1] # no padding: drop K//2 border → (H-2, W-
2)
stride = valid[:, ::2, ::2] # stride 2 (filter2D has NO strides arg -
manual is wrong)
true_conv = cv2.filter2D(gray, cv2.CV_32F, cv2.flip(k, -1)) # real
convolution = flipped kernel
# tested: 400x600 → same 400x600, valid 398x598, stride-2 199x299

def conv2d(im, k, stride=1, pad=0): # manual (if asked to implement from
scratch)
    im = np.pad(im.astype(np.float32), pad)
    kh, kw = k.shape
    oh = (im.shape[0] - kh)//stride + 1; ow = (im.shape[1] - kw)//stride + 1
    out = np.zeros((oh, ow), np.float32)
    for i in range(oh):
        for j in range(ow):
            out[i, j] = np.sum(im[i*stride:i*stride+kh, j*stride:j*stride+kw]
* k)
    return out

```

4C — Gradients: Sobel, Scharr, Prewitt

```

gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)      # dx=1 → vertical edges
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)      # dy=1 → horizontal
edges
mag, ang = cv2.cartToPolar(gx, gy, angleInDegrees=True) # = sqrt(gx2+gy2),
atan2; ang 0..360
mag_u8 = cv2.convertScaleAbs(mag)                    # abs + clip to uint8
for display
_, edge_map = cv2.threshold(mag_u8, 50, 255, cv2.THRESH_BINARY)

scharr_x = cv2.Scharr(gray, cv2.CV_64F, 1, 0)
prewitt_x = cv2.filter2D(gray, cv2.CV_64F, np.array([[ -1,0,1],[ -1,0,1],
[ -1,0,1]], np.float32))
show([gray, cv2.convertScaleAbs(gx), cv2.convertScaleAbs(gy), mag_u8],
      ['Gray', 'Sobel x', 'Sobel y', 'Magnitude'], cols=4)

```

4D — Feature descriptors

```

# HOG
feat, hog_img = hog(gray, orientations=9, pixels_per_cell=(8, 8),
                    cells_per_block=(2, 2), visualize=True)
hog_img = exposure.rescale_intensity(hog_img, in_range=(0, 10))
print('HOG vector length:', feat.shape)

# LBP
P, R = 8, 1
lbp = local_binary_pattern(gray, P, R, method='uniform')
lbp_hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, P + 3), range=(0, P
+ 2))
lbp_hist = lbp_hist / lbp_hist.sum()                # normalised, P+2 = 10
bins

# Colour histogram (HSV, 8x8 bins over H, S)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
chist = cv2.calcHist([hsv], [0, 1], None, [8, 8], [0, 180, 0, 256])
chist = cv2.normalize(chist, chist).flatten()        # 64-D

# HED: angles at edge pixels only
edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 0), 30, 70)
hed, _ = np.histogram(ang[edges > 0], bins=36, range=(0, 360))
# HIG: all pixels, weighted by magnitude

```

```

hig, _ = np.histogram(ang, bins=36, range=(0, 360), weights=mag)

# Texture energy & contrast (n x n neighbourhood)
n = 5; f = gray.astype(np.float32) # float! uint8 ** 2
overflows
energy = cv2.filter2D(f**2, -1, np.ones((n, n), np.float32) / (n*n))
mean = cv2.blur(f, (n, n))
contrast = np.sqrt(np.maximum(cv2.blur(f**2, (n, n)) - mean**2, 0))

# Compare two feature histograms
sim = cv2.compareHist(np.float32(h1), np.float32(h2), cv2.HISTCMP_CORREL) #
1 = identical
plt.bar(range(len(lbp_hist)), lbp_hist); plt.title('LBP histogram');
plt.show()

```

4E — Canny

```

blur = cv2.GaussianBlur(gray, (5, 5), 1.4)
edges = cv2.Canny(blur, 50, 150) # (img, low, high)
for lo, hi in [(50, 100), (50, 150), (50, 250)]: # threshold study: fix
    low, raise high
    pass # collect cv2.Canny(blur, lo, hi) and show()
v = np.median(gray) # auto thresholds
auto = cv2.Canny(blur, int(max(0, 0.67*v)), int(min(255, 1.33*v)))

```

4-TRAPS

- `cv2.filter2D` has **no `strides` argument** (manual's code raises an error, tested). Slice the output instead.
- Manual labels `BORDER_REFLECT` as zero padding — wrong. Zero padding = `BORDER_CONSTANT`. Default border = `BORDER_REFLECT_101`.
- Sobel with `-1` or `CV_8U` depth **loses negative gradients** (half the edges). Use `CV_64F`, then `convertScaleAbs`.
- `getGaussianKernel` returns a 1-D column ($k \times 1$); outer product `k @ k.T` gives 2-D.
- Square uint8 overflows: cast to float before `**2`.
- HOG/LBP need **grayscale** input. LBP 'uniform' gives $P+2$ bins.

Sheet 5 — Segmentation (Lab 5)

Needs: starter cell. 5B defines `coins / cg` ; 5E reloads its own image, so each block runs alone.

5-THEORY

- **Segmentation:** partition image into regions $R_1 \dots R_n$ such that $\cup R_i = \text{whole image}$, $R_i \cap R_j = \emptyset$, each R_i is connected, a predicate $P(R_i)$ is TRUE inside a region and FALSE for the union of two adjacent regions. Two principles: **similarity** (thresholding, region growing, clustering) and **discontinuity** (edge detection).
- Uses: medical (tumours), autonomous driving (road/pedestrian), satellite (land use), industrial inspection, document scanning.

Method	Assumption about the image	Fails when	Key parameter
Global threshold	Object & background separable by ONE intensity; uniform lighting	Uneven illumination, shadows	T
Adaptive threshold	Illumination varies slowly; local T = local mean (or Gaussian-weighted mean) – C	Large flat regions; block too small/large	blockSize (odd), C
Otsu	Histogram is bimodal (2 classes)	Unimodal histogram, small object, heavy noise	none (auto); pre-blur helps
Colour (HSV)	Target has a distinct hue	Similar-coloured background, shadows/highlights	lower/upper H, S, V
Canny / edges	Boundaries = sharp intensity changes	Weak/blurry boundaries, texture noise; edges not closed regions	low, high
Region growing	Region is connected & homogeneous around a seed	Leaks through weak boundaries; seed in wrong region	seed, threshold

Method	Assumption about the image	Fails when	Key parameter
Watershed	Objects are blobs; touching objects separated by distance-transform ridges	Over-segmentation without markers; irregular shapes	dist-transform fraction
K-means	Regions = colour clusters	Need K in advance; ignores spatial connectivity	K
Deep learning (U-Net, Mask R-CNN)	Large labelled data	No data	—
Active contours (snakes)	Smooth closed boundary near init	Bad initialisation, concave shapes	α, β, γ

- **Global:** $g = 255$ if $f > T$ else 0. **Adaptive:** $T(x,y) = \text{mean of blockSize} \times \text{blockSize neighbourhood} - C$. Small block $\rightarrow$ noisy, broken characters/texture amplified; large block $\rightarrow$ behaves like global (fails on gradient). Larger C $\rightarrow$ fewer pixels pass as "foreground" in BINARY $\rightarrow$ cleaner but thinner/broken text. Gaussian method weights centre pixels more $\rightarrow$ usually smoother.
- **Otsu:** tries every T, picks the one that **maximises between-class variance** $\sigma^2_B = w_0 \cdot w_1 \cdot (\mu_0 - \mu_1)^2$ (equivalently minimises within-class variance). Returned `ret` = chosen T. Useful when T is unknown or changes per image.
- **OpenCV HSV ranges:** H 0–179 (hue°/2), S 0–255, V 0–255. Approx hues: red 0–10 & 170–179, orange 10–20, yellow 20–35, green 35–85, blue 100–130, purple 130–160. Too-restrictive range $\rightarrow$ holes/missed parts (shading changes S and V).
- **Hysteresis (Canny):** strong > high kept; weak between kept only if connected to strong.
- **Region growing:** start at seed, add 4/8-neighbours whose $|I - I_{\text{seed}}| \leq T$. Different seed $\rightarrow$ different start intensity & connected component $\rightarrow$ different region. Larger T $\rightarrow$ leaks into neighbours.
- **Watershed:** image as topographic surface; flood from **markers**; dams = boundaries (marked -1). Without markers $\rightarrow$ over-segmentation. **Distance transform** = each foreground pixel's distance to nearest background; coin centres are peaks. Threshold too low $\rightarrow$ touching coins share one peak region (merged); too high $\rightarrow$ small objects lose markers or get split.

- **K-means:** pick K centres → assign each pixel to nearest centre → recompute centres → repeat until converged. Pixel = feature vector (B, G, R). Larger K → closer to original, less simplification.

5A — Setup

```
import cv2, numpy as np, matplotlib.pyplot as plt
gray = cv2.imread('sudoku.png', cv2.IMREAD_GRAYSCALE)
```

5B — Global, adaptive, Otsu

```
# Global with several T
outs = [cv2.threshold(gray, T, 255, cv2.THRESH_BINARY)[1] for T in [80, 127, 170]]
# THRESH_BINARY_INV for dark objects on light background

# Adaptive parameter grid (blockSize must be ODD and > 1)
res, titles = [gray], ['Original']
for method, name in [(cv2.ADAPTIVE_THRESH_MEAN_C, 'Mean'),
                     (cv2.ADAPTIVE_THRESH_GAUSSIAN_C, 'Gauss')]:
    for bs in [3, 11, 51]:
        for C in [2, 10]:
            res.append(cv2.adaptiveThreshold(gray, 255, method,
            cv2.THRESH_BINARY, bs, C))
            titles.append(f'{name} b={bs} C={C}')
show(res, titles, cols=4)

# Otsu (pass 0 as T; read the chosen T from ret)
coins = cv2.imread('coins.jpg'); cg = cv2.cvtColor(coins, cv2.COLOR_BGR2GRAY)
ret, otsu = cv2.threshold(cg, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print('Otsu T =', ret)
ret2, otsu2 = cv2.threshold(cv2.GaussianBlur(cg, (5, 5), 0), 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)      #
after preprocessing
low_c = cv2.convertScaleAbs(cg, alpha=0.6, beta=30)                    #
change contrast, repeat

plt.figure(figsize=(15, 4))
plt.subplot(131); plt.imshow(cg, cmap='gray'); plt.title('Original');
plt.axis('off')
plt.subplot(132); plt.hist(cg.ravel(), bins=256, range=(0, 256));
plt.axvline(ret, color='r'); plt.title(f'Histogram, T={ret}')
```

```
plt.subplot(133); plt.imshow(otsu, cmap='gray'); plt.title('Otsu');
plt.axis('off'); plt.show()

def otsu_manual(g):
    # if asked to implement (matches
    # OpenCV ±1)
    p = np.bincount(g.ravel(), minlength=256) / g.size
    best, bestT = 0, 0
    for t in range(1, 256):
        w0, w1 = p[:t].sum(), p[t:].sum()
        if w0 == 0 or w1 == 0: continue
        m0 = (np.arange(t) * p[:t]).sum() / w0
        m1 = (np.arange(t, 256) * p[t:]).sum() / w1
        v = w0 * w1 * (m0 - m1) ** 2 # between-class variance
        if v > best: best, bestT = v, t
    return bestT
```

5C — Colour (HSV) segmentation

```
img = cv2.imread('smarties.png'); hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
print(cv2.cvtColor(np.uint8([[0, 200, 255]]), cv2.COLOR_BGR2HSV)) # find
HSV of a BGR colour → H=24
# or sample the image: print(hsv[y, x])

restrictive = cv2.inRange(hsv, np.array([22, 200, 200]), np.array([26, 255,
255]))
mask = cv2.inRange(hsv, np.array([20, 100, 100]), np.array([35, 255,
255])) # yellow
k = np.ones((5, 5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k) # remove specks
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k) # fill holes
extracted = cv2.bitwise_and(img, img, mask=mask)

red = cv2.bitwise_or(cv2.inRange(hsv, (0, 70, 50), (10, 255, 255)), # red
wraps around!
cv2.inRange(hsv, (170, 70, 50), (179, 255, 255)))
show([img, restrictive, mask, extracted], ['Original', 'Too strict', 'Final
mask', 'Extracted'], cols=4)
```

5D — Region growing

```
def region_growing(img, seed, thresh):    # img gray, seed = (x, y)
    h, w = img.shape
    mask = np.zeros((h, w), np.uint8)
    visited = np.zeros((h, w), bool)
    seed_val = int(img[seed[1], seed[0]]) # int() avoids uint8 wrap-around
    stack = [seed]
    while stack:
        x, y = stack.pop()
        if x < 0 or y < 0 or x >= w or y >= h or visited[y, x]: continue
        visited[y, x] = True
        if abs(int(img[y, x]) - seed_val) <= thresh:
            mask[y, x] = 255
            stack.extend([(x+1, y), (x-1, y), (x, y+1), (x, y-1)]) # add
diagonals for 8-conn
    return mask

mri = cv2.imread('brain.jpg', 0)
masks = [region_growing(mri, (120, 140), T) for T in [10, 20, 40]] #
threshold study
masks += [region_growing(mri, s, 20) for s in [(80, 100), (150, 160)]] #
seed study
# Overlay: out = cv2.cvtColor(mri, cv2.COLOR_GRAY2BGR); out[mask == 255] =
(0, 0, 255)
```

5E — Watershed (10 stages, touching coins)

```
img = cv2.imread('coins.jpg'); gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0) #
1 preprocess
_, thresh = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
# 2 threshold
# coins must be WHITE: if they come out black use THRESH_BINARY_INV +
THRESH_OTSU
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2) # 3
noise removal
sure_bg = cv2.dilate(opening, kernel, iterations=3) # 4
sure background
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5) # 5
distance transform
```

```

_, sure_fg = cv2.threshold(dist, 0.7 * dist.max(), 255, 0) # 6
sure_foreground
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg) # 7
unknown region
n, markers = cv2.connectedComponents(sure_fg) # 8
label markers
markers = markers + 1 # background = 1, not 0
markers[unknown == 255] = 0 # 0 = let watershed decide
markers = cv2.watershed(img, markers) # 9
watershed (img = 3-ch BGR)
out = img.copy(); out[markers == -1] = [0, 0, 255] #
10 boundaries red
print('Coins:', n - 1) # minus background label

dist_vis = cv2.normalize(dist, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
show([img, thresh, sure_bg, dist_vis, sure_fg, unknown, out],
      ['Original', 'Threshold', 'Sure BG', 'Distance', 'Sure FG', 'Unknown',
'Watershed'], cols=4)

# Task 8 table: tested on 5 touching coins → 0.3: 3, 0.5: 3 (merged), 0.7: 5,
0.9: 5
print('Exp | Frac | Markers | Regions')
for i, f in enumerate([0.3, 0.5, 0.7, 0.9], 1):
    _, fg = cv2.threshold(dist, f * dist.max(), 255, 0); fg = np.uint8(fg)
    k, mk = cv2.connectedComponents(fg)
    mk = mk + 1; mk[cv2.subtract(sure_bg, fg) == 255] = 0
    mk = cv2.watershed(img.copy(), mk)
    print(i, f, k - 1, len(np.unique(mk)) - 2) # exclude -1 and background
(1)

```

5F — K-means

```
img = cv2.imread('dog.jpeg')
Z = img.reshape(-1, 3).astype(np.float32)          # (pixels, 3), float32
REQUIRED
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
segs = [img]
for K in [2, 4, 6]:
    compactness, labels, centers = cv2.kmeans(Z, K, None, criteria, 10,
cv2.KMEANS_RANDOM_CENTERS)
    seg = np.uint8(centers[labels.flatten()]).reshape(img.shape)    # each
pixel → its centre colour
    segs.append(seg)
    print(K, 'centres (BGR):', np.uint8(centers).tolist())
show(segs, ['Original', 'K=2', 'K=4', 'K=6'], cols=4)
# Single cluster mask: (labels.reshape(img.shape[:2]) == 0).astype(np.uint8)
* 255
```

5-TRAPS

- adaptiveThreshold **blockSize must be odd**; input must be 1-channel uint8.
- Otsu: pass `T = 0`; the real threshold is `ret . Otsu`. Otsu + adaptive cannot be combined.
- OpenCV hue is **0-179**, not 0-360. Red needs two ranges.
- Watershed needs a **3-channel** image and **int32** markers (connectedComponents gives int32). Boundary = -1. Forgetting `markers + 1` makes the background "unknown".
- If the threshold makes objects black, `sure_bg/sure_fg` are inverted → use `THRESH_BINARY_INV`.
- K-means data must be **float32**, shape (N, 3). Results vary per run (random init); `attempts=10` reduces this.
- Region growing: cast to `int` before subtracting, else uint8 wraps (`10 - 20 = 246`).

Sheet 6 — Edges, Hough, SIFT, wavelets (Lab 6)

Needs: starter cell + 6A (adds `import pywt`). 6D's video version reuses the loop from 2G.

6-THEORY

Wavelets

- **Wavelet:** small, finite-duration wave. Unlike Fourier (global sine waves, frequency only), wavelets give **time/space AND frequency** localisation → multi-resolution analysis.

Type	What it is	Use
CWT (continuous)	All scales & shifts; redundant; Morlet, Mexican hat	Time-frequency analysis, oscillations
DWT (discrete)	Dyadic scales (powers of 2); efficient, non-redundant; Haar, Daubechies (db), Symlet	Compression (JPEG2000), denoising, feature extraction
WPT (packet)	Splits BOTH approximation and detail at every level	Finer frequency resolution
Inverse WT	Rebuilds signal from coefficients	Reconstruction after thresholding

- **DWT 1-D:** each level → approximation **cA** (low-pass, trend) + detail **cD** (high-pass, noise/edges); cA is decomposed again. **2-D DWT** → LL (approx) + LH (horizontal), HL (vertical), HH (diagonal) detail, each half size.
- **Denoising:** decompose → threshold detail coefficients (soft: shrink toward 0; hard: zero below T) → reconstruct. Universal threshold $T = \sigma \cdot \sqrt{2 \cdot \ln N}$, $\sigma = \text{median}(|cD1|)/0.6745$.
- **Anomaly detection:** residual = signal – denoised; flag points where $|\text{residual}| > k \cdot \text{std}$ ($k \approx 3$).

Edges (Lab 6 part 3)

- **Sobel** = 1st derivative (gradient, "speed"), edges at peaks. **Laplacian** = 2nd derivative ("acceleration"), edges at **zero-crossings**, isotropic, very noise-sensitive. **LoG** = Gaussian smoothing first, then Laplacian (fixes noise sensitivity). **Canny** = smooth → gradient → NMS → hysteresis.

Hough transform (Paul Hough, 1962)

- Maps edge points from image space to **parameter space**; each edge point **votes** in an **accumulator**; peaks = detected shapes. Robust to gaps and noise. Needs an edge (binary) image first.
- **Lines:** $y = mx + c$ fails for vertical lines ($m = \infty$), so use **$p = x \cdot \cos\theta + y \cdot \sin\theta$** (p = perpendicular distance from origin, θ = angle of the normal). One point → a **sinusoid** in (p, θ) space; collinear points' sinusoids intersect at one (p, θ) .

- `HoughLines` (standard) returns (ρ, θ) , infinite lines. `HoughLinesP` (probabilistic) returns segment endpoints, faster. Params: ρ resolution (1 px), θ resolution ($\pi/180$), threshold (min votes), `minLineLength`, `maxLineGap`.
- **Circles:** $(x - a)^2 + (y - b)^2 = r^2 \rightarrow$ 3-D parameter space (a, b, r) . OpenCV `HOUGH_GRADIENT` uses gradient direction to cut cost. Params: `dp` (accumulator resolution inverse ratio), `minDist` (between centres), `param1` (Canny high threshold), **`param2` (accumulator threshold: lower = MORE circles, incl. false ones)**, `minRadius`, `maxRadius`.

SIFT (Lowe, 2004) — invariant to scale, rotation; partially to illumination & viewpoint.

1. **Scale-space extrema detection:** Gaussian pyramid $\rightarrow$ Difference of Gaussians (DoG, approximates LoG); each pixel compared with 26 neighbours (8 same scale + 9 above + 9 below).
 2. **Keypoint localisation:** sub-pixel refinement (Taylor expansion); reject low-contrast points and **edge** responses (Hessian ratio).
 3. **Orientation assignment:** gradient orientation histogram (36 bins) around keypoint; dominant peak (plus peaks $> 80\%$) $\rightarrow$ rotation invariance.
 4. **Descriptor:** 16×16 window $\rightarrow 4 \times 4$ subregions $\times 8$ orientation bins = **128-D vector**, normalised $\rightarrow$ illumination robust.
 5. **Matching:** nearest neighbour by **Euclidean (L2) distance**; **Lowe's ratio test:** keep if $d_1 < 0.75 \cdot d_2$ (best clearly better than second best).
 6. **Outlier rejection / homography:** **RANSAC** picks random 4-match subsets, fits H , counts inliers, keeps the best $\rightarrow$ geometrically consistent matches only. Needs ≥ 4 matches (use ≥ 10 in practice).
- **Panorama:** SIFT keypoints in both images $\rightarrow$ match + ratio test $\rightarrow$ homography (RANSAC) $\rightarrow$ warp one image into the other's plane $\rightarrow$ overlay/blend $\rightarrow$ crop black borders.

6A — Sobel magnitude, Canny, Laplacian / LoG

```
import cv2, numpy as np, pywt, matplotlib.pyplot as plt
img = cv2.imread('image.jpg'); gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3); sy = cv2.Sobel(gray,
cv2.CV_64F, 0, 1, ksize=3)
mag = np.uint8(np.clip(np.sqrt(sx**2 + sy**2), 0, 255))
blur = cv2.GaussianBlur(gray, (5, 5), 1.4)
canny = cv2.Canny(blur, 50, 150)
log_ = cv2.convertScaleAbs(cv2.Laplacian(blur, cv2.CV_64F, ksize=3)) # LoG
= blur + Laplacian
show([gray, mag, canny, log_], ['Gray', 'Sobel', 'Canny', 'LoG'], cols=4)
```

6B — Hough lines (lanes, screens)

```
edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 0), 50, 150)

# Lanes: keep only a road-shaped region of interest
h, w = edges.shape
roi = np.zeros_like(edges)
poly = np.array([[0, h), (w//2 - 50, h//2 + 20), (w//2 + 50, h//2 + 20), (w,
h)], np.int32)
cv2.fillPoly(roi, poly, 255)
masked = cv2.bitwise_and(edges, roi)

lines = cv2.HoughLinesP(masked, 1, np.pi/180, threshold=50, minLineLength=40,
maxLineGap=20)
out = img.copy()
if lines is not None: # ALWAYS check None
    for x1, y1, x2, y2 in lines[:, 0]:
        slope = (y2 - y1) / (x2 - x1 + 1e-6)
        if abs(slope) < 0.5: continue # lanes: drop near-
horizontal lines
        cv2.line(out, (x1, y1), (x2, y2), (0, 0, 255), 3)
# Screens: keep near-horizontal (abs(slope)<0.1) AND near-vertical
(abs(slope)>10) lines instead

# Standard Hough → (rho, theta) → draw long line
L = cv2.HoughLines(edges, 1, np.pi/180, 120)
if L is not None:
```

```

for rho, theta in L[:, 0]:
    a, b = np.cos(theta), np.sin(theta); x0, y0 = a*rho, b*rho
    cv2.line(out, (int(x0 - 1000*b), int(y0 + 1000*a)), (int(x0 +
1000*b), int(y0 - 1000*a)), (255, 0, 0), 2)

```

6C — Hough circles (coin counting)

```

coins = cv2.imread('coins.jpg')
cg = cv2.medianBlur(cv2.cvtColor(coins, cv2.COLOR_BGR2GRAY), 5)    # gray +
blur, NOT Canny
circles = cv2.HoughCircles(cg, cv2.HOUGH_GRADIENT, dp=1.2, minDist=50,
                           param1=100, param2=20, minRadius=20, maxRadius=60)

out = coins.copy()
if circles is not None:
    circles = np.uint16(np.around(circles))
    for x, y, r in circles[0]:
        cv2.circle(out, (x, y), r, (0, 255, 0), 2)        # outline
        cv2.circle(out, (x, y), 2, (0, 0, 255), 3)        # centre
    n = len(circles[0])
    cv2.putText(out, f'Coins: {n}', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1,
(255, 0, 0), 2)
# Tested: 5 touching coins → param2=30 found 2, param2=20 found all 5. Tune
param2 first.

```

6D — SIFT keypoints, matching, object recognition

```

sift = cv2.SIFT_create()                                # optional:
SIFT_create(nfeatures=500)
kp, des = sift.detectAndCompute(gray, None)              # des: (N, 128)
float32
img = cv2.drawKeypoints(gray, kp, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
print('Keypoints:', len(kp), 'Descriptor shape:', des.shape)

ref = cv2.imread('object.jpg', 0)                       # reference object
test = cv2.imread('test.jpg'); tg = cv2.cvtColor(test, cv2.COLOR_BGR2GRAY)
kp1, d1 = sift.detectAndCompute(ref, None)
kp2, d2 = sift.detectAndCompute(tg, None)

bf = cv2.BFMatcher(cv2.NORM_L2)                          # L2 for SIFT
(NORM_HAMMING for ORB)
matches = bf.knnMatch(d1, d2, k=2)

```

```

good = [m for m, n in matches if m.distance < 0.75 * n.distance]      # Lowe
ratio test
mimg = cv2.drawMatches(ref, kp1, tg, kp2, good[:50], None,
                        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

if len(good) >= 10:                                                    # bounding box via
homography
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, inliers = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    hh, ww = ref.shape
    corners = np.float32([[0, 0], [ww, 0], [ww, hh], [0, hh]]).reshape(-1, 1,
2)
    box = cv2.perspectiveTransform(corners, H)
    found = cv2.polylines(test.copy(), [np.int32(box)], True, (0, 255, 0), 3)
    print('Object found, inliers:', int(inliers.sum()))
else:
    print('Object not found')
# Tested on rotated + scaled scene: 122 good matches, 115 RANSAC inliers.
# FLANN alternative: cv2.FlannBasedMatcher(dict(algorithm=1, trees=5),
dict(checks=50)).knnMatch(d1, d2, k=2)
# Video: wrap the test part in the VideoCapture loop (Sheet 2G), compute
kp/des of ref ONCE outside.
# Asset tracking: one ref per asset; the asset with the most good matches =
identified.

```

6E — Panorama stitching

```
def stitch(img1, img2):                                # img2 is to the
RIGHT of img1
    sift = cv2.SIFT_create()
    g1, g2 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY), cv2.cvtColor(img2,
cv2.COLOR_BGR2GRAY)
    k1, d1 = sift.detectAndCompute(g1, None); k2, d2 =
sift.detectAndCompute(g2, None)
    good = [m for m, n in cv2.BFMatcher().knnMatch(d2, d1, k=2) if m.distance
< 0.75 * n.distance]
    src = np.float32([k2[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
# img2 points
    dst = np.float32([k1[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
# img1 points
    H, _ = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
# img2 → img1
    pano = cv2.warpPerspective(img2, H, (img1.shape[1] + img2.shape[1],
max(img1.shape[0], img2.shape[0])))
    pano[:img1.shape[0], :img1.shape[1]] = img1
    x, y, w, h = cv2.boundingRect(cv2.findNonZero(cv2.cvtColor(pano,
cv2.COLOR_BGR2GRAY)))
    return pano[y:y+h, x:x+w]                        # crop black area

pano = stitch(cv2.imread('left.jpg'), cv2.imread('right.jpg'))
# 3+ images: pano = stitch(stitch(a, b), c)
# Built-in check (status 0 = OK): status, p =
cv2.Stitcher_create().stitch([a, b, c])
```

6F — Wavelets: denoise, anomalies, 2-D DWT

```
import pandas as pd
# data = pd.read_csv('sensor.csv')['value'].values      # real data
np.random.seed(0)
t = np.linspace(0, 10, 1000)
signal = np.sin(2*np.pi*t) + 0.3*np.random.randn(1000)
signal[[200, 550, 800]] += [4, -3.5, 5]              # injected anomalies

coeffs = pywt.wavedec(signal, 'db4', level=4)         # [cA4, cD4, cD3,
cD2, cD1]
sigma = np.median(np.abs(coeffs[-1])) / 0.6745
thr = sigma * np.sqrt(2 * np.log(len(signal)))        # universal
```

```

threshold
den = [coeffs[0]] + [pywt.threshold(c, thr, mode='soft') for c in coeffs[1:]]
denoised = pywt.waverec(den, 'db4')[:len(signal)]      # trim: length can
grow by 1
residual = signal - denoised
anomalies = np.where(np.abs(residual) > 3 * np.std(residual))[0]  # tested:
finds 200, 550, 800

plt.figure(figsize=(12, 6))
plt.subplot(211); plt.plot(t, signal, label='Sensor'); plt.plot(t, denoised,
label='Denoised')
plt.legend(); plt.title('Sensor Data and Denoised Signal')
plt.subplot(212); plt.plot(t, residual, label='Residuals')
plt.scatter(t[anomalies], residual[anomalies], color='r', label='Anomalies')
plt.legend(); plt.title('Residuals and Detected Anomalies');
plt.tight_layout(); plt.show()

# 2-D DWT on an image
cA, (cH, cV, cD) = pywt.dwt2(gray, 'haar')            # each half size
rec = pywt.idwt2((cA, (cH, cV, cD)), 'haar')          # perfect
reconstruction (tested)
show([cA, np.abs(cH), np.abs(cV), np.abs(cD)], ['LL approx', 'LH horiz', 'HL
vert', 'HH diag'], cols=4)

# CWT and wavelet packet
cwt_coef, freqs = pywt.cwt(signal, np.arange(1, 32), 'morl')
plt.imshow(np.abs(cwt_coef), aspect='auto', cmap='jet'); plt.title('CWT
scalogram'); plt.show()
wp = pywt.WaveletPacket(signal, 'db1', maxlevel=2)
print([node.path for node in wp.get_level(2, 'natural')])  # ['aa', 'ad',
'da', 'dd']
# families: pywt.families() → haar, db, sym, coif, ... ; pywt.wavelist('db')

```

6G — Contours, boundaries, security zone

```

_, th = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)      # or Canny
edges
cnts, _ = cv2.findContours(th, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
out = img.copy()
cv2.drawContours(out, cnts, -1, (0, 255, 0), 2)                # -1 = all
contours

```

```

for c in cnts:
    rectangles (screens)
    approx = cv2.approxPolyDP(c, 0.02 * cv2.arcLength(c, True), True)
    if len(approx) == 4 and cv2.contourArea(c) > 1000:
        x, y, w, h = cv2.boundingRect(approx)
        cv2.rectangle(out, (x, y), (x+w, y+h), (0, 0, 255), 2)

# Security zone on video
zone = np.array([[100, 100], [500, 100], [500, 350], [100, 350]], np.int32)
bgs = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=50,
detectShadows=True)
cap = cv2.VideoCapture('cam.mp4')
while True:
    ret, frame = cap.read()
    if not ret: break
    fg = bgs.apply(frame)
    fg = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY)[1] # drop
    shadows (127)
    fg = cv2.morphologyEx(fg, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
    cnts, _ = cv2.findContours(fg, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    cv2.polylines(frame, [zone], True, (255, 0, 0), 2)
    for c in cnts:
        if cv2.contourArea(c) < 500: continue
        x, y, w, h = cv2.boundingRect(c); cx, cy = x + w//2, y + h//2
        if cv2.pointPolygonTest(zone, (float(cx), float(cy)), False) >= 0:
# ≥0 inside/on edge
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 0, 255), 2)
            cv2.putText(frame, 'ALARM', (10, 40), cv2.FONT_HERSHEY_SIMPLEX,
1.2, (0, 0, 255), 3)
            cv2.imshow('Security', frame)
            if cv2.waitKey(30) & 0xFF == ord('q'): break
    cap.release(); cv2.destroyAllWindows()
# Static alternative: diff = cv2.absdiff(background_gray, frame_gray) →
threshold → contours

```

6-TRAPS

- `HoughLines`, `HoughLinesP`, `HoughCircles` return **None** when nothing is found → check before looping.
- `HoughLines/LinesP` need a **binary edge image**. `HoughCircles` needs a **blurred grayscale** image (it runs Canny internally with param1).

- HoughCircles: lower **param2** → more circles (false positives). Coins missing → lower param2 or fix min/maxRadius.
- SIFT: `cv2.SIFT_create()` (OpenCV ≥ 4.4; old `xfeatures2d` not needed). Use **NORM_L2**, not Hamming. `findHomography` needs ≥ 4 points.
- `queryIdx` indexes the FIRST descriptor set passed to `knnMatch`, `trainIdx` the second.
- `waverec` output can be 1 sample longer → slice `[:len(signal)]`.
- `findContours` modifies nothing in OpenCV 4 and returns 2 values (contours, hierarchy).